EE P 596 Computer Vision — Assignment #4 Report

CIFAR-10 classification with a CNN | University of Washington

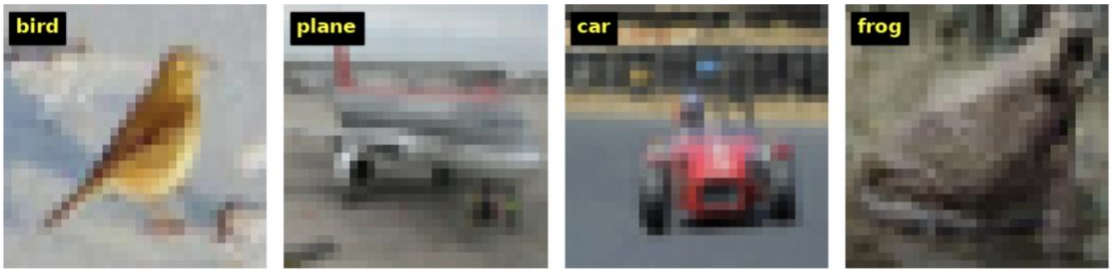
1. CIFAR-10 dataset

Values from cifar10/cifar-10-batches-py/readme.html and the downloaded files:

Name	Value
num_train_batches	5
num_test_batches	1
num_img_per_batch	10,000
num_train_img	50,000
num_test_img	10,000
size_batch_bytes	≈ 30,308 KB (data_batch_1 file on disk)
size_image_bytes	3 KB (32 × 32 × 3 = 3,072 bytes)
size_batchimage_bytes	30,000 KB (10,000 × 3,072 = 30,720,000 bytes)

(a) One random mini-batch of 4 training images

```
images: torch.Size([4, 3, 32, 32]) torch.FloatTensor
labels: tensor([2, 0, 1, 6]) ['bird', 'plane', 'car', 'frog']
```



Random mini-batch of 4 training images with ground-truth labels overlaid (plt.text)

2. Train classifier

Step 1 – Network output on a mini-batch

The untrained network maps a batch of 4 images to an output of shape (4, 10): for each image, one score per CIFAR-10 class. The class with the highest score is the prediction.

```
output shape: torch.Size([4, 10])
```

Step 2 – Training (2 epochs, cross-entropy loss, SGD lr = 0.001, momentum = 0.9)

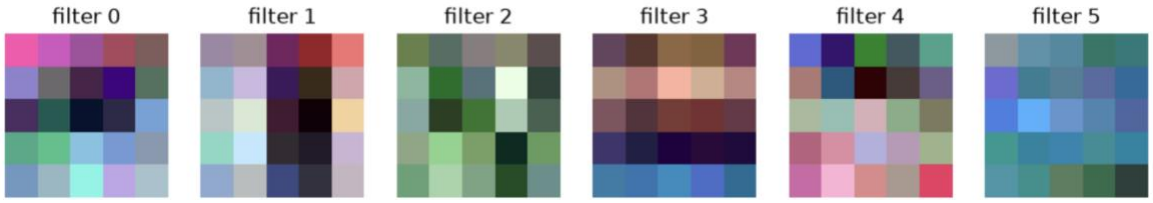
The training loss decreased steadily, from about 2.21 after the first 2,000 mini-batches to about 1.30 at the end of epoch 2, comparable to the PyTorch tutorial. The weights were saved as cifar_net_2epoch.pth.

Step 3 – Test accuracy (weights loaded from cifar_net_2epoch.pth)

```
Accuracy on the 10,000 test images: 53.27 %
```

3. Visualize weights

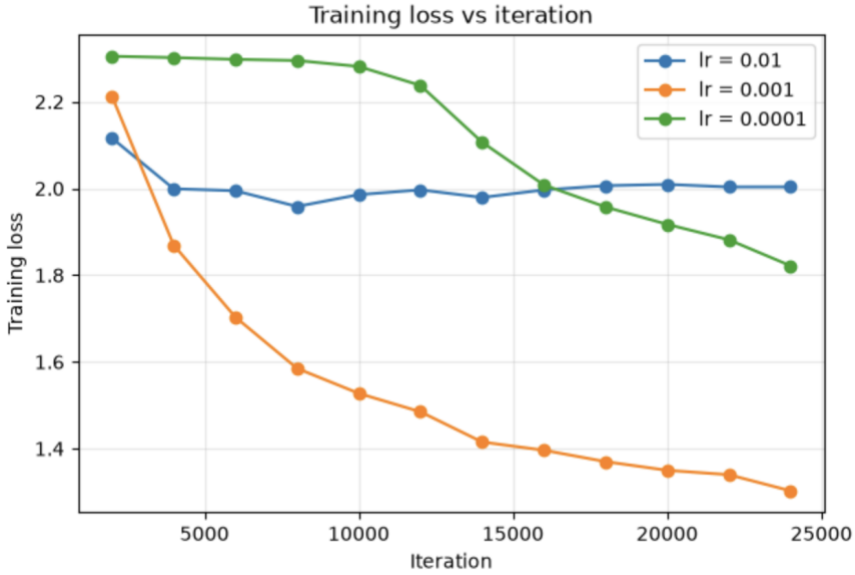
```
conv1 weights: torch.Size([6, 3, 5, 5])
conv2 weights: torch.Size([16, 6, 5, 5])
```



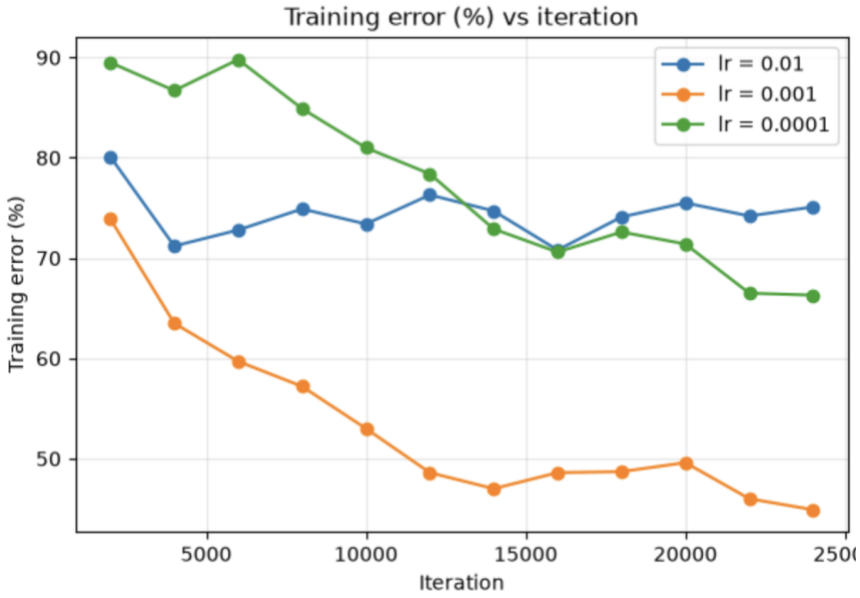
The 6 learned conv1 filters (5 × 5 × 3), each min–max scaled for display

4. Hyperparameter sweep

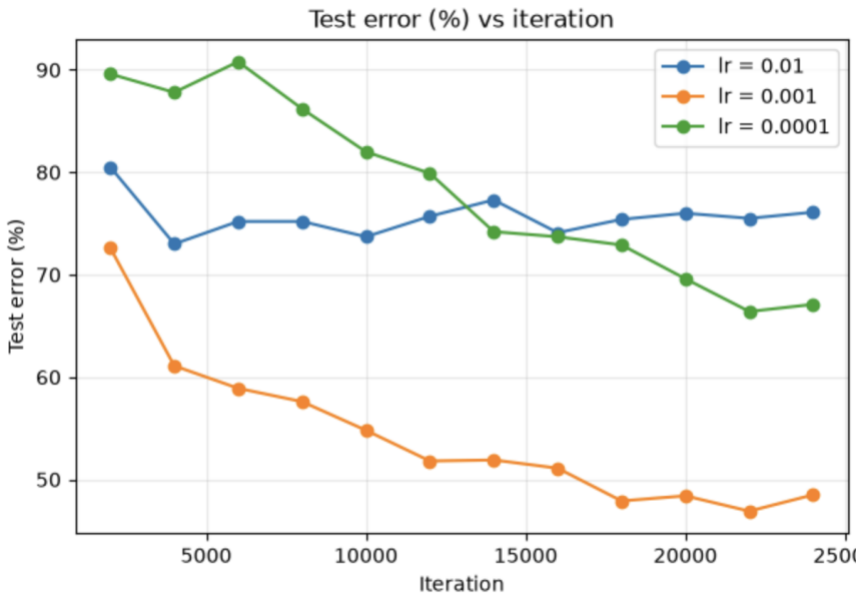
The network was retrained for 2 epochs with learning rates 0.01, 0.001 and 0.0001. Every 2,000 iterations the average training loss was recorded, and the training and test errors were computed on 1,000 randomly sampled images from each set.



(i) Training loss vs iteration



(ii) Training error vs iteration



(iii) Test error vs iteration

Learning rate	Final training loss	Final training error	Final test error
0.01	2.003	75.1 %	76.1 %
0.001	1.302	44.9 %	48.5 %
0.0001	1.822	66.3 %	67.1 %

(d) Discussion

The learning rate strongly affected training. With lr = 0.001, the training loss decreased steadily from about 2.21 to 1.30, and both training and test error fell from about 73% to roughly 45–48%, giving the best result of the three. With lr = 0.01, the step size was too large: the loss dropped slightly at first but then stayed flat around 2.0 and the errors remained near 75%, so the network stopped improving. With lr = 0.0001, the step size was too small: the loss stayed almost unchanged (about 2.30) for the first 10,000 iterations, then began to decrease slowly, reaching 1.82 with about 66–67% error by the end of 2 epochs; it was still improving but would need many more epochs. In all three cases training and test error stayed close together, so there was little overfitting in 2 epochs. Overall, lr = 0.001 gave the best balance of speed and stability.